

AWK & GREP: TECHNICAL GUIDE

1. AWK Arithmetic and Logical Operators

In AWK, operators are used to perform calculations and make decisions based on conditions.

Arithmetic Operators in AWK

Arithmetic operators perform mathematical calculations like addition, subtraction, multiplication, etc.

Syntax:

```
awk '{ print expression }' filename
or
awk '{ print $field operator value }' filename
```

Operator	Meaning	Example
+	Addition	\$2+10
-	Subtraction	\$2-5
*	Multiplication	\$2*2
/	Division	\$2/2
%	Modulus (Remainder)	\$2%2

++	Increment	i++
--	Decrement	i--

Example File (marks.txt)

```
Rahul 80  
Amit 70  
Priya 90
```

Example 1: Addition (+)

Add 5 bonus marks.

```
awk '{print $1,$2+5}' marks.txt
```

Output:

```
Rahul 85  
Amit 75  
Priya 95
```

Explanation: \$1 = Name, \$2 = Marks, +5 adds bonus marks.

Example 2: Subtraction (-)

```
awk '{print $1,$2-10}' marks.txt
```

Output:

```
Rahul 70  
Amit 60  
Priya 80
```

Example 3: Multiplication (*)

```
awk '{print $1,$2*2}' marks.txt
```

Output:

```
Rahul 160  
Amit 140  
Priya 180
```

Example 4: Division (/)

```
awk '{print $1,$2/2}' marks.txt
```

Output:

```
Rahul 40  
Amit 35  
Priya 45
```

Example 5: Modulus (%)

Find whether marks are even or odd.

```
awk '{print $1,$2%2}' marks.txt
```

Output:

```
Rahul 0  
Amit 0  
Priya 0
```

If the remainder is 0, the number is even.

Example 6: Increment (++)

```
awk 'BEGIN{  
i=1  
print i  
i++  
print i  
'
```

Output:

```
1  
2
```

Example 7: Decrement (--)

```
awk 'BEGIN{  
i=5  
print i  
i--  
print i  
'
```

Output:

```
5  
4
```

Real-World Example (Arithmetic)

Suppose a company gives every employee a ₹1000 bonus.

File:

```
Rahul 30000  
Amit 25000  
Priya 40000
```

Command:

```
awk '{print $1,$2+1000}' salary.txt
```

Output:

```
Rahul 31000  
Amit 26000  
Priya 41000
```

2. Logical Operators in AWK

Logical operators are used to combine two or more conditions. They return either True (1) or False (0).

Syntax:

```
awk 'condition1 logical_operator condition2 {action}' filename
```

Operator	Meaning
&&	Logical AND
	Logical OR
!	Logical NOT

Example File (employee.txt)

```
101 Rahul IT 50000
102 Amit HR 30000
103 Priya IT 70000
104 Riya Sales 45000
```

Columns: \$1 = ID, \$2 = Name, \$3 = Department, \$4 = Salary

Logical AND (&&)

Both conditions must be true. Show employees who work in IT and earn more than ₹60,000.

```
awk '$3=="IT" && $4>60000 {print $2,$4}' employee.txt
```

Output:

```
Priya 70000
```

Logical OR (||)

At least one condition must be true. Show employees from HR or employees earning more than ₹60,000.

```
awk '$3=="HR" || $4>60000 {print $2,$3,$4}' employee.txt
```

Output:

```
Amit HR 30000  
Priya IT 70000
```

Logical NOT (!)

Reverses the condition. Display employees not in the IT department.

```
awk '!( $3=="IT" ) {print $2,$3}' employee.txt
```

Output:

```
Amit HR  
Riya Sales
```

Combined Example

Display employees whose salary is greater than ₹40,000 and who are not in the HR department.

```
awk '$4>40000 && !($3="HR") {print $2,$3,$4}' employee.txt
```

Output:

```
Rahul IT 50000  
Priya IT 70000  
Riya Sales 45000
```

Real-World Example (Logical Operators)

Suppose a college wants to list students who scored more than 80 marks and belong to the CS department.

File:

```
Rahul CS 85  
Amit EE 90  
Priya CS 95  
Riya CS 75
```

Command:

```
awk '$2="CS" && $3>80 {print $1,$3}' students.txt
```

Output:

```
Rahul 85  
Priya 95
```


Exam Quick Revision

Arithmetic Operators

Operator	Use	Example
+	Addition	\$2+10
-	Subtraction	\$2-5
*	Multiplication	\$2*2
/	Division	\$2/2
%	Remainder	\$2%2
++	Increment	i++
--	Decrement	i--

Logical Operators

Operator	Meaning	Example
&&	AND (both conditions true)	\$3=="IT" && \$4>60000
	OR (at least one condition true)	\$3=="HR" \$4>60000
!	NOT (reverse the condition)	!(\$3=="IT")

GREP COMMAND AND TASK

What is grep?

grep stands for **Global Regular Expression Print**.

It is a UNIX command used to search text inside one or more files. **Important:** grep only reads a file. It does not modify the original file.

Syntax:

```
grep [options] "pattern" filename
```

Example File: employee.txt

```
101 Alice HR Delhi
102 Bob IT Mumbai
103 Charlie Sales Kolkata
104 David IT Chennai
105 Emma HR Delhi
106 Bob Sales Delhi
107 Frank Finance Mumbai
108 alice IT Pune
109 George HR Kolkata
110 Bob HR Delhi
```

1. Simple Search

```
grep "Bob" employee.txt
```

Output:

```
102 Bob IT Mumbai  
106 Bob Sales Delhi  
110 Bob HR Delhi
```

2. Ignore Case (-i)

```
grep -i "alice" employee.txt
```

Output:

```
101 Alice HR Delhi  
108 alice IT Pune
```

3. Show Line Number (-n)

```
grep -n "Bob" employee.txt
```

Output:

```
2:102 Bob IT Mumbai  
6:106 Bob Sales Delhi  
10:110 Bob HR Delhi
```

4. Count Matching Lines (-c)

```
grep -c "Bob" employee.txt
```

Output:

```
3
```

5. Invert Match (-v)

```
grep -v "Bob" employee.txt
```

Output: (Lines not containing Bob)

```
101 Alice HR Delhi
103 Charlie Sales Kolkata
104 David IT Chennai
105 Emma HR Delhi
107 Frank Finance Mumbai
108 alice IT Pune
109 George HR Kolkata
```

6. Match Whole Word (-w)

```
grep -w "cat" file.txt
```

7. Exact Entire Line (-x)

```
grep -x "Bob" file.txt
```

8. Search Multiple Files

```
grep "Delhi" employee.txt student.txt
```

9. Recursive Search (-r)

```
grep -r "main" project/
```

10. Show Only File Name (-l)

```
grep -l "Delhi" *.txt
```

11. Show Files Without Match (-L)

```
grep -L "Delhi" *.txt
```

12. Search Beginning of Line (^)

```
grep "^Bob" file.txt
```

13. Search End of Line (\$)

```
grep "Delhi$" employee.txt
```

14. Multiple Patterns (-e)

```
grep -e "Bob" -e "Alice" employee.txt
```

15. Search with Wildcard

```
grep "HR" *.txt
```

16. Search Using Pipe

```
ls -l | grep "employee"
```

Summary of GREP

Does grep Modify the File?

No. grep is a search command, not an editing command. It only displays matching lines on the terminal.

Most Important grep Flags for Exams

Flag	Meaning	Example
-i	Ignore uppercase/lowercase	grep -i "alice" employee.txt
-n	Show line numbers	grep -n "Bob" employee.txt
-v	Show non-matching lines	grep -v "Bob" employee.txt
-c	Count matching lines	grep -c "Bob" employee.txt
-w	Match whole word only	grep -w "cat" file.txt
-x	Match the entire line	grep -x "Bob" file.txt
-l	Show filenames with pattern	grep -l "Delhi" *.txt
-L	Show filenames without pattern	grep -L "Delhi" *.txt
-r	Search recursively	grep -r "main" project/
-e	Multiple patterns	grep -e "Bob" -e "Alice"

Real-Life Example

Imagine a company has a file with 10,000 employee records, and the manager wants to find all employees working in the IT department. Instead of checking every line manually, they can run:

```
grep "IT" employee.txt
```

This instantly displays only the matching records, saving time and effort. The original file remains exactly the same.